

In this code we create an instance of the `Example` class, make a call to `print()`, and execute a simple assignment. From the client's point of view, this code is highly readable. When we write `oExample.x`, it looks like we are using direct access to an instance variable. However, when client code accesses the value of an object's property (on the right side of an assignment statement or as an argument in a call to a function or method), Python translates it into a call to the getter method of the object. When an object dot property appears on the left side of an assignment statement, Python calls the setter method. The getter and setter methods affect the real instance variable, `self._x`.

Here is a more realistic example that should help make this clear. [Listing 8-6](#) shows a `Student` class that includes a property `grade`, properly decorated getter and setter methods, and a private instance variable `__grade`.

File: PropertyDecorator/Student.py

```
# Using a property to (indirectly) access data in an object

class Student():

    def __init__(self, name, startingGrade=0):
        self.__name = name
        self.grade = startingGrade ❶

    @property ❷
    def grade(self): ❸
        return self.__grade

    @grade.setter ❹
    def grade(self, newGrade): ❺
        try:
            newGrade = int(newGrade)
        except (TypeError, ValueError) as e:
            raise type(e)('New grade: ' + str(newGrade) + ', is an invalid
type.')
        if (newGrade < 0) or (newGrade > 100):
            raise ValueError('New grade: ' + str(newGrade) + ', must be between 0
and 100.')
        self.__grade = newGrade
```
